

Evidence log

What I observed, when, and with what values. Session on 15 August 2026 against `http://149.56.128.57:3200`, signed in as `member@nexapay.test` (Kwame Mensah, role Member) unless noted.

The screenshots that go with this are produced by `npm run evidence`, which writes full-page captures to `evidence/screenshots/` and attaches them to the HTML report. I could not generate them inside my own sandbox because the Chromium download is blocked there, so they are produced on first run.

Login screen

Two fields and a button, no data-testid attributes on any of them. Placeholders read “Enter your email” and “Enter your password”, button reads “Sign In”. The app redirects an unauthenticated visitor to `#/login?returnUrl=%2Fdashboard`, so the routing is hash-based. That matters: the plain paths I assumed in round one do not resolve.

Dashboard, clean state

Signed in as Member. Toast reads “Welcome back, Kwame Mensah!”.

Card	Value
Total Balance	\$48,291.00 (+2.4%)
Income	\$12,750.00 (+5.1%)
Expenses	\$8,420.00 (-1.8%)
Savings Rate	33.9% (+0.7%)

Sidebar shows Dashboard, Transactions, Transfer, Upload. No Admin entry, which is correct for this role.

Against the API at the same moment: 15 transactions summing to $-\$389.66$, or $\$1,883.14$ counting completed only. Income across positive amounts is $\$9,052.49$, expenses across negative amounts $\$9,442.15$. None of the four cards matches.

The Savings Rate is internally consistent with the two cards next to it: $(12750 - 8420) / 12750 = 33.96\%$. So that block computes correctly from numbers that are themselves disconnected from the ledger.

Dashboard, polluted state

This one was an accident and it turned out to be the most useful measurement I took. After my eight API test transfers, one of which was for a million dollars, the same card read:

Card	Value
Total Balance	-\$100,010,822.00
Income	\$12,750.00, unchanged
Expenses	\$8,420.00, unchanged

The chart’s Y axis rescaled to $\$120,000,000$.

Two conclusions. Total Balance responds to the data, so it is derived, which disproves the “hard-coded” claim I made in my first write-up. Income and Expenses did not move at all despite 23 extra transactions in the database, so those two do look static.

After deleting the test data the balance returned to \$48,291.00, but only after a hard reload. Navigating between pages left the stale figure on screen. A financial dashboard showing an old balance after the data has changed is a problem in itself.

Transactions list

Item	Value
Balance shown	\$50,563.80
Rows per page	5
Pagination	"Page 1 of 3"
Total records	15, matching the API

So the dashboard says \$48,291.00 and this screen says \$50,563.80, a difference of \$2,272.80, for the same user at the same moment.

First page, sorted by date descending:

Date	Description	Category	Amount	Status
Jul 22, 2025	ATM Withdrawal, Mall branch	Withdrawal	-\$300.00	Completed
Jul 20, 2025	Utility Payment, Electric bill	Payment	-\$215.30	Pending
Jul 18, 2025	Insurance Refund, Policy #INS-442	Refund	+\$620.00	Completed
Jul 17, 2025	Transfer to Sarah Chen, Gift	Transfer	-\$150.00	Failed
Jul 15, 2025	Freelance Income, Design work	Payment	+\$2,800.00	Completed

Every row matches the API record exactly. Rendering is fine.

Worth noting against Annexe F.6, which documents dates in August 2026 and labels like "Loyer juillet". The deployed data is July 2025 with entirely different labels. The documentation is stale relative to what is actually running.

Status filters

Filter	Rows	Pagination	Balance
All	15	Page 1 of 3	\$50,563.80
Completed	10	Page 1 of 2	\$50,563.80
Pending	3	Page 1 of 1	\$50,563.80
Failed	2	Page 1 of 1	\$50,563.80

Row counts match the API exactly, so filtering works.

The balance not moving is expected behaviour, not a defect. Filters change which rows are displayed; an account balance is computed over the whole dataset. I originally read this as proof the figure was hard-coded, which it is not, and the polluted-state measurement above is what corrected me.

Transfer form

Hooks present: transfer-recipient, transfer-amount, transfer-note, transfer-type with -instant, -standard and -scheduled variants, transfer-pin, transfer-submit. The type control is an Angular Material radio group, so the testid sits on the host element and the checkable input is inside it.

Validation, with amount 10000 and PIN 12345:

- “Amount cannot exceed \$9,999.99” appears. R03 holds in the form.
- “PIN must be exactly 6 digits” appears. R02 holds in the form.
- The note counter tracks correctly, “25/60 characters”.
- Clicking submit in this state sends no request. The form does block itself.

The submit button is not visually disabled while the form is invalid, which is poor affordance, but the handler does guard.

Transfer submission

Recipient “Sarah Chen”, amount 12.34, note “QA critical journey TS-05”, type Instant selected, PIN 123456. No validation errors on screen, submit enabled.

Clicking it produced nothing. No network request, checked by wrapping both fetch and XMLHttpRequest before clicking. No toast, no navigation, no error, nothing in the console. Repeated with a native click() on the element, same result.

Admin route as a Member

Navigating to #/admin directly redirects to #/dashboard. The guard works.

API surface

Path	Result
/db	200, entire database
/transactions	200, 15 records
/api/v1/transactions	200, identical to the above
/users	200, 3 records, no auth header sent
/contacts	200, 3 records, all owners
/transfers	200, empty array in clean state
/health, /ready, /metrics	404

The app itself calls /transactions?_sort=date&_order=desc.

Transfer POSTs

Eight requests, all 201.

Body	Result
Valid, with Idempotency-Key	201, id 1
Same body, same key again	201, id 2
amount: 1000000000	201, id 3
amount: -2500	201, id 4
No pin field	201, id 5
scheduledDate: "2020-01-01"	201, id 6
61-character note	201, id 7
{"hello": "world"}	201, id 8

Stored record, verbatim:

```
{
  "recipientId": "c1",
  "currency": "USD",
```

```
"transferType": "instant",
"pin": "123456",
"amount": 1234,
"note": "QA TS-05",
"id": 1,
"createdAt": "2026-08-15T10:42:55.070Z"
}
```

The PIN is there in clear. The id is an integer where the contract says UUID. There is no status field, which the contract says there should be.

Each POST also created a matching transaction record, described as “Transfer to undefined”, so there is some custom server code beyond a plain fixture. It still validates nothing.

Browser environment

```
window.isSecureContext    false
location.protocol         http:
window.crypto.subtle      undefined
window.crypto.randomUUID  undefined
```

R08 requires the PIN to be SHA-256 hashed in the browser. The Web Crypto API is only available in a secure context, so on this deployment it cannot be called at all.

Cleanup

Testing added 23 transactions and 16 transfer records to the shared demo. All removed. Final state verified: 15 transactions, ids tx-001 to tx-015, summing to -\$389.66, transfers collection empty, dashboard reading \$48,291.00 again.

DELETE also works with no authentication, so any candidate can wipe this environment. Mine was recoverable only because the original records use tx-0NN ids and I could tell them apart from what I had added.